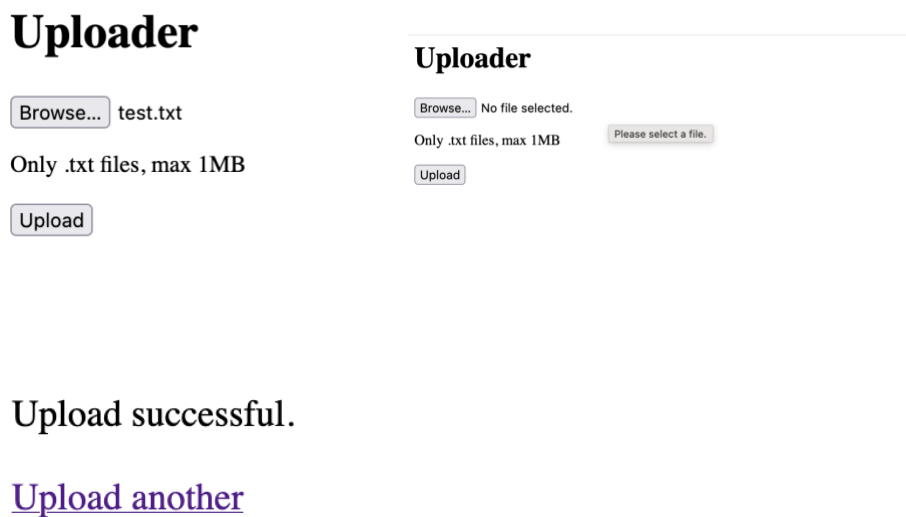


1. Screenshot of your local lab03/uploader-app/ and lab03/viewer-app/ folders showing app.js and package.json in each.

```
root@8d53bccd4076:/# ls -l lab03/uploader-app
[ls -l lab03/viewer-app
total 8
-rw-r--r-- 1 root root 1862 Oct  3 17:48 app.js
-rw-r--r-- 1 root root  167 Oct  3 17:43 package.json
total 8
-rw-r--r-- 1 root root 927 Oct  3 18:31 app.js
-rw-r--r-- 1 root root 139 Oct  3 17:56 package.json
root@8d53bccd4076:/#
```

```
Launching instances from the templates
Waiting for both instances to reach 'running'...
-----
| DescribeInstances |
+-----+
| uploader-app | 18.221.0.52 |
| viewer-app   | 3.15.203.54 |
+-----+
Instances are running. Give them ~60-90s to finish installing Node.js before browsing to :3000.
root@8d53bccd4076:/#
```

2. Screenshot of the Uploader page and a successful upload confirmation.

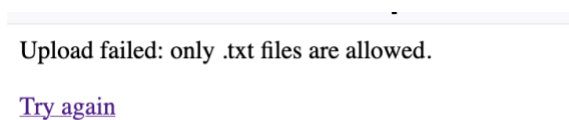


3. Screenshot of attempting to upload a .txt file larger than 1MB, showing the “file exceeds the 1MB limit” error.

Upload failed: file exceeds the 1MB limit.

[Try again](#)

4. Screenshot of attempting to upload a non- .txt file (e.g., a .jpg or .pdf), showing the “only .txt files are



allowed” error.

4. Screenshot of the Viewer page displaying the uploaded file’s content.



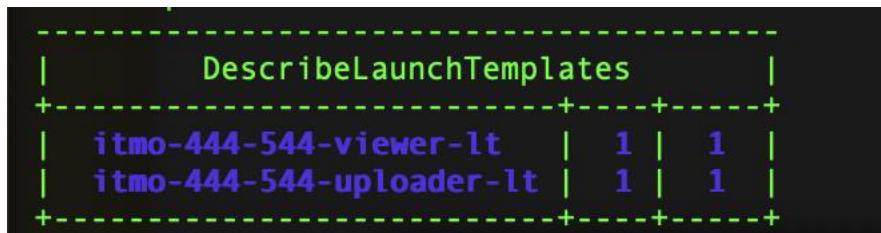
6. Screenshot of the Viewer page after uploading a second, different file and clicking Refresh, showing the updated content.

Viewer

2nd upload

[Refresh](#)

7. Screenshot of aws ec2 describe-launch-templates showing both templates.



8. Screenshot of aws ec2 describe-launch-template-versions --launch-template-name <uploader-lt-name> after re-running the create script twice, showing more than one version.

```
[ --output table
+-----+
| DescribeLaunchTemplateVersions |
+-----+
| 3 | 2026-10-03T21:35:21+00:00 |
| 2 | 2026-10-03T21:34:09+00:00 |
| 1 | 2026-10-03T21:05:08+00:00 |
+-----+
```

9. aws iam get-role-policy output (or screenshot) for both roles, showing each has only its one intended action on

the exact shared.txt key.

```
--role-name itmo-444-544-uploader-role \
[ --policy-name itmo-444-544-uploader-role-policy
{
  "RoleName": "itmo-444-544-uploader-role",
  "PolicyName": "itmo-444-544-uploader-role-policy",
  "PolicyDocument": {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": [
          "s3:PutObject"
        ],
        "Resource": "arn:aws:s3:::itmo-444-544-jcrosby2-fileshare/shared.txt"
      }
    ]
  }
}
```

```
--role-name itmo-444-544-viewer-role \
[ --policy-name itmo-444-544-viewer-role-policy
{
  "RoleName": "itmo-444-544-viewer-role",
  "PolicyName": "itmo-444-544-viewer-role-policy",
  "PolicyDocument": {
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": [
          "s3:GetObject"
        ],
        "Resource": "arn:aws:s3:::itmo-444-544-jcrosby2-fileshare/shared.txt"
      },
      {
        "Effect": "Allow",
        "Action": [
          "s3:ListBucket"
        ],
        "Resource": "arn:aws:s3:::itmo-444-544-jcrosby2-fileshare"
      }
    ]
  }
}
```

10. Screenshot of create_app_stack.sh output showing both instances running with public IPs.

```

[root@8d53bccd4076:/# ./create_app_stack.sh
== S3 bucket ==
{
  "BucketArn": "arn:aws:s3:::itmo-444-544-jcrosby2-fileshare",
  "BucketRegion": "us-east-2",
  "AccessPointAlias": false
}
Bucket already exists, skipping.
== Uploader role (PutObject on shared.txt only) ==
Role exists, skipping.
Profile
exists, skipping.
== Viewer role (GetObject + ListBucket on shared.txt only) ==
Role exists, skipping.
Profile
exists, skipping.
Waiting 15s for IAM propagation...
== Building user-data from local lab03/ files ==
== Building launch template data ==
== Creating/updating launch templates ==
Template itmo-444-544-uploader-lt exists - publishing a new version
Template itmo-444-544-viewer-lt exists - publishing a new version
== Launching instances from the templates ==
Waiting for both instances to reach 'running'...
-----
|           DescribeInstances           |
+-----+-----+
| uploader-app | 3.144.253.185 |
| viewer-app   | 3.16.130.140  |
+-----+-----+
Instances are running. Give them ~60-90s to finish installing Node.js before browsing to :3000.

```

11. Screenshot of delete_app_stack.sh completing with no errors.

```

Terminating instances: i-0720a2902112b8cc1 i-082f58b47fb3a76e0
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-0720a2902112b8cc1",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    },
    {
      "InstanceId": "i-082f58b47fb3a76e0",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
Instances terminated.
== Deleting launch templates ==
{
  "LaunchTemplate": {
    "LaunchTemplateId": "lt-09a1840776f7525c0",
    "LaunchTemplateName": "itmo-444-544-uploader-lt",
    "CreateTime": "2026-10-03T21:05:08+00:00",
    "CreatedBy": "arn:aws:iam::916609685464:user/lab_user",
    "DefaultVersionNumber": 1,
    "LatestVersionNumber": 4,
    "Operator": {
      "Managed": false
    }
  }
}
{
  "LaunchTemplate": {
    "LaunchTemplateId": "lt-0cf11a8544195afd0",
    "LaunchTemplateName": "itmo-444-544-viewer-lt",
    "CreateTime": "2026-10-03T21:05:09+00:00",
    "CreatedBy": "arn:aws:iam::916609685464:user/lab_user",
    "DefaultVersionNumber": 1,
    "LatestVersionNumber": 4,
    "Operator": {
      "Managed": false
    }
  }
}
}
== Removing IAM instance profiles and roles ==
== Emptying and deleting S3 bucket ==
delete: s3://itmo-444-544-jcrosby2-fileshare/shared.txt
Cleanup complete.

```

12. One paragraph explaining why the uploader and viewer use separate IAM roles instead of one shared role with both permissions.

The uploader and viewer use different roles so that they are using the principle of least privilege and utilize only the permissions they needed to allow them to perform their task. The uploader needs `s3:PutObject` to upload files to the S3 bucket. The viewer uses `s3:GetObject` permission to read files. By using strict permissions this prevents further compromise so that an attacker cannot modify or overwrite files and instead only read them. Assigning different roles helps to reduce the risk of unauthorized actions and maintain better control over access to the S3 bucket.

13. One or two sentences explaining why embedding code directly into user-data means the instance never needs a Git or S3 credential just to fetch its own code.

By embedding directly into the EC2, user data can be used to configure data automatically during startup without having to download it from git or S3. The instance does not need git credentials or s3 access permissions to pull application code because it is already listed in the startup instructions.